VERIQTA

# AWS INCIDENT RESPONSE HANDBOOK

**DETECT | INVESTIGATE | CONTAIN | RECOVER | PREVENT**




@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

# AWS INCIDENT RESPONSE ARCHITECTURE

## 1. What is AWS Incident Response Architecturer?

- A structured approach to detect, investigate, contain, recover and prevent incidents in AWS.
- It defines roles, ownership, tooling, communication and the architecture needed for fast and safe incident response.
- It covers infrastructure, applications, security, data and dependencies across accounts and regions.

## 2. Incident Lifecycle

| | |
|---|---|
| Detect | Identify abnormal behaviour using metrics, logs, alerts and security findings. |
| Triage | Validate the issue, assess impact and determine severity. |
| Contain | Limit blast radius and stop further damage. |
| Eradicate | Remove the root cause. |
| Recover | Restore full service and verify stability. |
| Review | Conduct a blameless postmortem and implement improvements. |

## 3. AWS Shared Responsibility Model

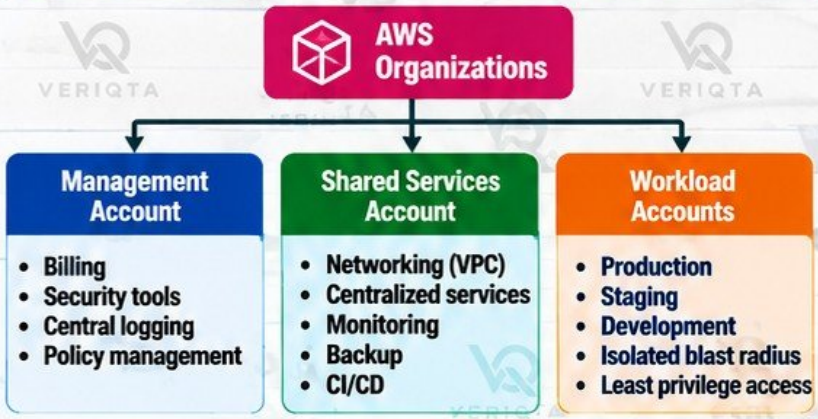
### AWS RESPONSIBILITY

- Global infrastructure
- Regions and Availability Zones
- Physical security
- Networking and hardware
- Compute, storage and database infrastructure
- Managed service availability
- Hypervisor and underlying systems

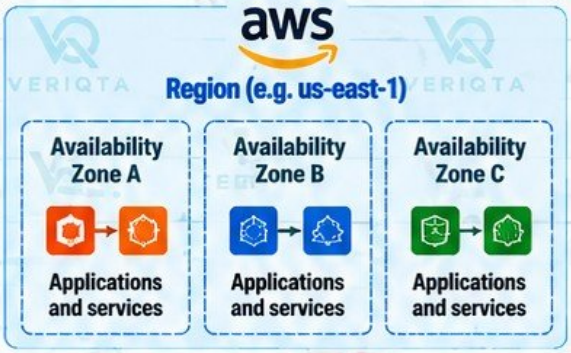
### YOUR RESPONSIBILITY

- Applications and data
- Identity and access management
- Network configuration
- Security groups and firewalls
- Operating systems and patches
- Data encryption and key management
- Monitoring, logging and alerting
- Incident response and recovery
- Compliance and governance

## 4. AWS Organizations and Account Isolation




## 5. Regional Failure Domains




- An Availability Zone (AZ) is an isolated data centre.
- A Region contains multiple AZs.
- Design for failure: assume an AZ can fail at any time.
- Use multi-AZ architectures to improve resilience.

## 6. Security vs Availability Incident Response

### Security Incidents

- Unauthorized access or data breach
- Compromised credentials
- Malicious activity or ransomware
- DDoS attacks
- Suspicious API activity (GuardDuty)
- Focus: contain threat, preserve evidence, maintain forensic data, then recover.

### Availability Incidents

- Service outages or degradation
- High error rates or latency
- Resource exhaustion (CPU, memory, etc.)
- Dependency failures
- Regional or AZ disruption
- Focus: restore service quickly, minimize customer impact, then investigate root cause.

## 7. Senior Engineer Decision

- Establish incident command immediately.
- Determine severity and customer impact.
- Identify the correct escalation path.
- Bring the right people together early (engineering, security, product, support).
- Avoid making changes without understanding the blast radius.
- Communicate clearly and frequently.
- Document decisions, timestamps and actions.

# BUILDING AN AWS INCIDENT DETECTION SYSTEM

## 1. CloudWatch Metrics, Logs, Alarms and Anomaly Detection

Amazon CloudWatch

- Collects metrics, logs and events from AWS services and applications.
- Create alarms for key indicators (CPU, memory, latency, error rate, throttling, queue depth, etc).
- Use anomaly detection to automatically detect unusual behaviour.
- Use dashboards for real-time visibility.
- Aggregate logs with CloudWatch Logs and use Logs Insights for investigation.

### Common Alerts to Monitor

- High error rate (4xx, 5xx)
- Increased latency
- Sudden drop in traffic
- CPU / memory saturation
- Throttling (e.g. API limits, Lambda)
- Queue backlog
- Database connections near limit
- Unhealthy targets in ALB
- Unexpected cost spikes

## 2. CloudTrail, AWS Config and EventBridge

### AWS CloudTrail

- Logs all AWS API activity (who did what, when).
- Helps detect unauthorized changes and security incidents.
- Monitor with CloudWatch and EventBridge.

### AWS Config

- Tracks resource configuration and changes.
- Detects drift and non-compliant resources.
- Use rules to alert on risky changes (e.g. public S3 bucket, security group open to 0.0.0.0/0).

### Amazon EventBridge

- React to changes in real time (e.g. CloudTrail events, Config rules, AWS Health).
- Trigger automated responses (SNS, Lambda, SSM, etc).
- Central event bus for cross-account monitoring.

### Example EventBridge Rule

Trigger an alert when a Security Group is opened to the internet.

```
{
  "source": ["aws.config"],
  "detail-type": ["Config Rules
   Compliance Change"],
  "detail": {
    "configRuleName": ["restricted-ssh"],
    "newEvaluationResult": {
     "complianceType": ["NON_COMPLIANT"]
    }
  }
}
```

## 3. GuardDuty, Security Hub and AWS Health

### Amazon GuardDuty

- Detects malicious activity and threats.
- Finds compromised credentials, unusual API activity and malware.
- Use findings to trigger automated response.

### Security Hub

- Centralizes security findings from AWS services and partners.
- Provides a unified view of security posture.
- Helps prioritize and respond to security incidents.

### AWS Health

- Provides real-time information about AWS service issues.
- Alerts on planned changes and service disruptions.
- Integrate with EventBridge to notify your team.

### AWS Health Event Types

- Service issue (availability or performance)
- Planned change (maintenance)
- Account notification (e.g. limits)
- Security notification
- Health event for specific region or service

## 4. Signal Correlation Across Services and Accounts

- CloudWatch (metrics, logs, alarms)
- CloudTrail (API activity)
- AWS Config (resource changes)
- GuardDuty (threats)
- AWS Health (service status)

→

### Aggregation and Correlation

- Use EventBridge, Security Hub and centralized logging.
- Correlate signals across multiple accounts and services.
- Identify patterns and related events.

→

### Detection and Alerting

- Create unified alerts for real customer impact.
- Reduce alert noise using correlation.
- Detect multi-layer incidents (e.g. network + application + database).

→

### Response and Automation

- Trigger automated notifications (SNS, Slack, PagerDuty, etc).
- Start automated runbooks (SSM, Lambda).
- Escalate to on-call team.
- Provide context and recommended actions.

## 5. Production Insight

**Detect customer impact before infrastructure symptoms become outages.**

- Monitor user-facing metrics first (latency, error rate, success rate, business KPIs).
- Set alerts based on customer experience, not just infrastructure thresholds.
- Use synthetic monitoring and real user monitoring (RUM) where possible.
- Correlate application-level signals with infrastructure metrics to detect issues early.
- Early detection gives you time to contain the issue before it affects many users.

# INCIDENT SEVERITY, TRIAGE, AND BLAST RADIUS

## 1. Incident Severity Classification (SEV1 – SEV4)

| Severity | Customer Impact | Description | Response |
|---|---|---|---|
| SEV1 | Major outage<br>No or severe degradation | Critical customer-facing service down or unusable. Significant business impact. | Immediate response 24/7 |
| SEV2 | Limited outage<br>Degraded service | Customer-facing service partially impacted. Important functionality unavailable. | High priority response |
| SEV3 | Minor impact<br>Non-critical | Minimal customer impact. Workaround available. Minor service degradation. | Normal response |
| SEV4 | No customer impact | Internal issue, maintenance or low-risk problem. | Standard process |

## 2. Customer-Facing vs Internal Failures

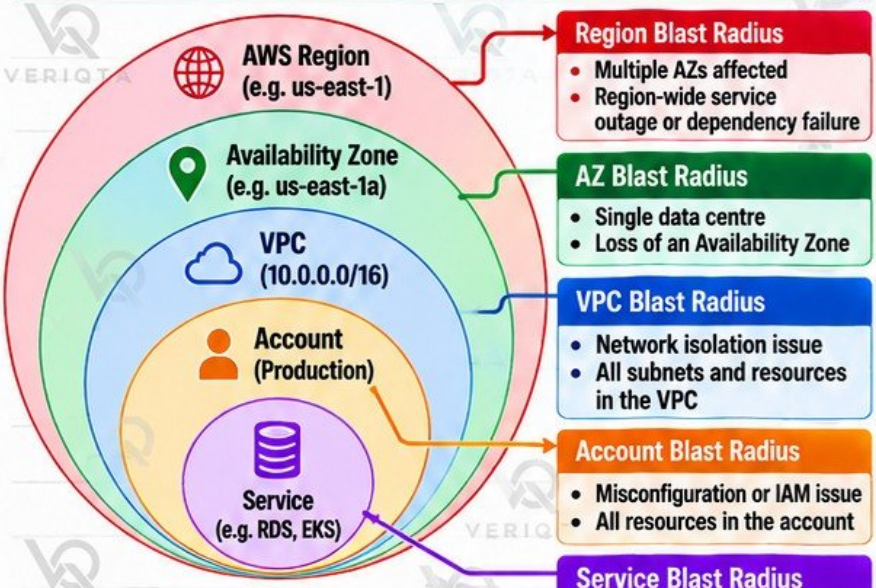
### Customer-Facing Failures

- Affects external users or paid customers.
- Direct business impact (revenue, reputation).
- Higher severity (often SEV1/SEV2).
- Examples:
  - Web application down
  - API returning 5xx errors
  - Checkout failures
  - Public website latency

### Internal Infrastructure Failures

- Affects internal teams, non-customer workloads or backend systems.
- Usually SEV3 or SEV4 unless it risks customer impact.
- Examples:
  - CI/CD pipeline failure
  - Monitoring system down
  - Internal database issue
  - Development environment outage

## 3. Understanding Blast Radius in AWS




### Region Blast Radius

- Multiple AZs affected
- Region-wide service outage or dependency failure

### AZ Blast Radius

- Single data centre
- Loss of an Availability Zone

### VPC Blast Radius

- Network isolation issue
- All subnets and resources in the VPC

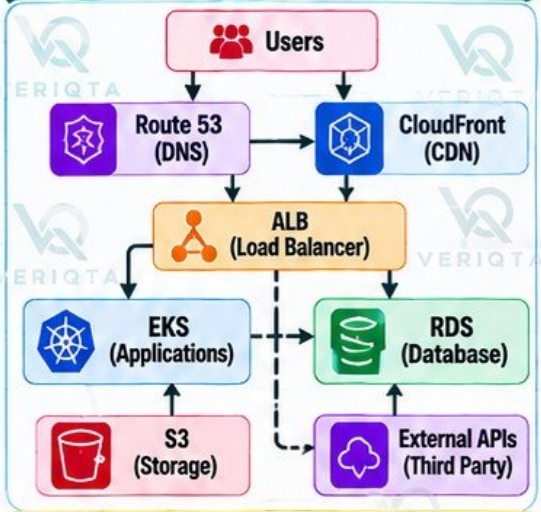
### Account Blast Radius

- Misconfiguration or IAM issue
- All resources in the account

### Service Blast Radius

- Single service failure
- e.g. RDS, S3, EKS, Lambda

## 4. Dependency Mapping and Recent-Change Correlation




**Key steps during triage:**

- Identify all dependencies (upstream and downstream).
- Check recent changes (deployments, config updates, scaling events).
- Correlate timelines across services (metrics, logs, CloudTrail, events).
- Determine the likely failure domain.

## 5. Triage Checklist

1. Confirm customer impact and severity.
2. Identify what is failing and since when.
3. Check recent changes (deployments, config, scaling).
4. Review metrics, logs, alarms and AWS Health.
5. Map dependencies and determine blast radius.
6. Form and test hypotheses.
7. Decide on containment actions.
8. Communicate status to stakeholders.

## 6. Senior Engineer Decision

- Prioritize containment and customer impact over premature root-cause claims.
- Make decisions based on evidence, not assumptions.
- Escalate early for SEV1 and SEV2.
- Avoid making large changes before understanding blast radius.
- Keep stakeholders informed with clear, factual updates.
- Focus on restoring service safely, then perform deeper root-cause analysis.

## 7. Common Mistakes to Avoid

- Underestimating customer impact.
- Jumping to a root cause too early.
- Making changes without understanding blast radius.
- Ignoring recent deployments or config changes.
- Failing to consider cross-account or cross-Region dependencies.
- Not communicating during an incident.
- Overloading teams with uncoordinated actions.

# EC2 FAILURE INVESTIGATION AND RECOVERY

## 1. EC2 Status Checks and Common Failure Types

### Status Checks

- **System status check**
  Underlying hardware, network and AWS platform.
- **Instance status check**
  OS level (CPU, memory, network, services).
- Both must be OK for the instance to be healthy.

### Common Failure Types

- Impaired host (hardware issue).
- OS/kernel issue.
- Application crash or misconfiguration.
- Network connectivity problem.
- Security group or NACL blocking traffic.
- Disk full or inode exhaustion.
- CPU or memory saturation.
- Service dependency failure.

## 2. EC2 Failure Investigation Flow

Check status checks
(EC2 Console / AWS CLI)

↓

Check metrics and logs
(CloudWatch)

↓

Check OS and application
(SSM / Serial Console)

↓

Identify root cause
(hardware, OS, app, network, dependency)

↓

Apply mitigation and verify
(service healthy, traffic restored)

## 3. Key Diagnostics and Commands

### Check instance status

```
aws ec2 describe-instance-status
--instance-ids i-0123456789abcdef0
```

- View system and instance status checks.
- Look for impaired reachability or failed checks.

### Check CloudWatch metrics

```
aws cloudwatch get-metric-statistics
--namespace AWS/EC2
--metric-name CPUUtilization
--dimensions Name=InstanceId,
  Value=i-0123456789abcdef0
```

- Check CPU, memory (if agent installed), disk, network metrics.

### Check disk space (via SSM or SSH)

```
df -h
df -i
```

- Identify disk or inode exhaustion.
- Look for /var or application log growth.

### Check top processes

```
top
ps aux --sort=-%mem | head
```

- Find high CPU or memory consumers.
- Check for runaway or zombie processes.

### Check recent system logs

```
sudo journalctl -xe --since "1 hour ago"
sudo dmesg | tail
```

- Look for OOM kills, kernel errors, disk or hardware issues.

### Use EC2 Serial Console (if no access)

```
aws ec2 get-console-output
--instance-id i-0123456789abcdef0
```

- Useful when the instance is unreachable.
- Check boot errors or kernel panics.

## 4. Recovery Options and Trade-Offs

| Option | When to Use | Advantages | Trade-Offs |
|---|---|---|---|
| Stop / Start | OS issue, temporary failure | Quick and simple | Downtime, may not fix root cause |
| Replace Instance | Hardware issue, repeated failures | Fresh instance, on healthy host | Needs new bootstrap, possible IP change |
| Rebuild from AMI | Corrupt OS, config issue | Clean, consistent instance | Time to rebuild and reattach resources |
| Auto Scaling Recovery | Stateless apps, multiple instances | Self-healing, maintains capacity | Requires correct health checks and configuration |
| Investigate Dependency | App working but no external connectivity | Fixes real cause | More time to identify and validate |

## 5. Common Metrics to Investigate

### CPU Utilization

- Sustained > 80%
- CPU credit exhaustion (t-family)

### Memory Utilization

- High usage
- OOM kills in logs

### Disk Utilization

- > 85% usage
- Check inode usage (df -i)

### EBS Latency

- High read/write latency
- Burst limits exceeded

### Network In/Out

- Unexpected spikes
- Packet loss or throttling

## 6. Production Scenario

**An EC2 instance passes infrastructure status checks but stops serving traffic.**

**Investigate:**

- Application logs and process status
- Security group and NACL rules
- Load balancer target health
- Recent deployments or configuration changes
- Dependency failures (database, API, etc.)

## 7. Senior Engineer Decision

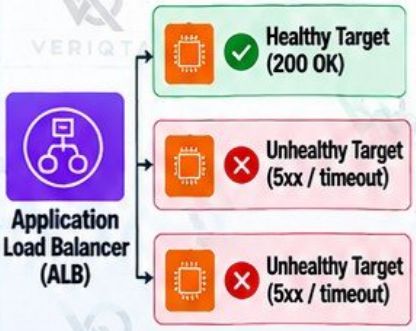
- Prioritize containment and customer impact over premature root-cause claims.
- Validate the actual impact and blast radius.
- Use metrics, logs and evidence to form hypotheses.
- Choose the safest recovery option based on the likely cause.
- After recovery, perform a full root-cause analysis and implement preventive actions.

## 8. Key Takeaways

- Always verify both system and instance status.
- Use CloudWatch, logs and OS diagnostics together.
- Don't rely only on status checks.
- Consider dependencies and network configuration.
- Automate recovery with Auto Scaling where appropriate.

# AUTO SCALING AND LOAD BALANCER INCIDENTS

## 1. Application Load Balancer (ALB) Target Health and 5xx Failures




- ALB sends health checks to targets (default / or custom path).
- Unhealthy targets are removed from the load balancer.
- HTTP 5xx, timeouts or high latency can cause targets to be marked unhealthy.
- Check target health, target response codes and ALB metrics (TargetResponseTime, HTTPCode_Target_5XX_Count).

## 2. Health Check Misconfiguration and Replacement Loops

### Common Misconfigurations

- Wrong health check path.
- Incorrect success codes.
- Health check timeout too short.
- Unrealistic threshold settings.
- Application not ready but returning 5xx.
- Security group or NACL blocking health check traffic.
- High latency causing false failures.

### Result

- Targets repeatedly marked unhealthy.
- ALB deregisters and re-registers targets.
- Auto Scaling keeps replacing instances.
- Creates a replacement loop and continuous errors.

### Health Check Flow

ALB health check (e.g. /health)
↓
Target returns 5xx or times out
↓
Marked unhealthy
↓
Target deregistered
↓
Auto Scaling launches new instance
↓
Loop repeats if issue not fixed

## 3. Auto Scaling Capacity, Warm-Up, Quotas and Scaling Lag

### Key Concepts

- Auto Scaling adjusts capacity based on metrics (CPU, request count, etc).
- Instance warm-up time delays when a new instance starts receiving traffic.
- Scaling can be delayed by EC2 limits, service quotas or slow bootstrap.
- New instances may fail health checks and never become healthy.
- Scaling out too quickly can overload downstream dependencies (database, cache, API).

### Important Metrics

- GroupDesiredCapacity
- GroupInServiceInstances
- GroupPendingInstances
- GroupTerminatingInstances
- CPUUtilization
- ALBRequestCount
- TargetResponseTime
- HTTPCode_Target_5XX_Count
- HealthyHostCount

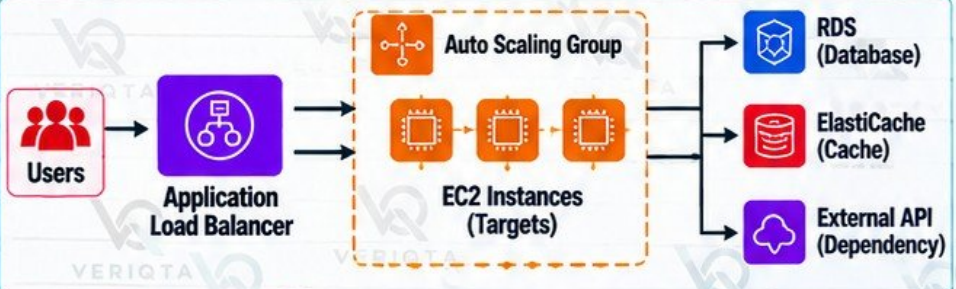
## 4. Target Deregistration and Connection Draining

- When a target is deregistered, ALB stops sending new requests to it.
- Existing connections continue until they finish (connection draining).
- Default deregistration delay is 300 seconds (configurable).
- Set appropriate delay for long-lived connections (e.g. WebSockets, large uploads).
- Ensure the application handles SIGTERM and graceful shutdown during scale in or deployments.

**AWS CLI: Configure deregistration delay**

```
aws elbv2 modify-target-group-attributes \
--target-group-arn arn:aws:elasticloadbalancing:region:123456789012:targetgroup/app/abc123 \
--attributes Key=deregistration_delay.timeout_seconds,Value=120
```

## 5. Architecture: ALB with Auto Scaling




## 6. Production Scenario

**Scaling activity increases error rates instead of restoring capacity.**

### Possible Causes

- New instances not passing health checks.
- Application not fully intialized (warm-up).
- Incorrect health check path or success codes.
- Downstream dependency saturation.
- Configuration drift (e.g. wrong security group).
- Auto Scaling launching instances in a constrained AZ.
- CPU or memory limits too low.

## 7. Troubleshooting Steps

1. Check ALB target health and reason for unhealthy status.
2. Review ALB metrics (5xx count, target response time).
3. Verify health check configuration and application startup time.
4. Check Auto Scaling events and instance lifecycle.
5. Inspect instance logs, CloudWatch metrics and user-data.
6. Check downstream dependencies for saturation.
7. Look for recent changes (deployments, config, scaling policies).

## 8. Senior Engineer Decision

- Prioritize customer impact and stabilize the service first.
- Validate health check configuration and application readiness.
- Consider scaling policy adjustments (warm-up, step size, cooldown).
- Avoid rapid scaling if dependencies are unhealthy.
- Use connection draining for graceful scale in.
- Increase instance size or optimize application if saturation persists.

## 9. Key Takeaways

- Validate application readiness before sending traffic.
- Use realistic health check configuration.
- Monitor both ALB and Auto Scaling metrics.
- Consider downstream dependency limits.
- Use connection draining to avoid user impact.
- Investigate recent changes and scaling events during incidents.

# ECS, EKS, AND CONTAINER FAILURE RESPONSE

## 1. ECS Task Failures and Deployment Instability

Amazon ECS

- Tasks can fail due to image, IAM, networking, resource limits or application errors.
- Service deployment can become unstable if new tasks fail health checks.
- Tasks may repeatedly start and stop (crash loop).
- Check task events, container logs and ECS service events.
- Use ECS deployment circuit breaker to stop failing deployments.

## 2. EKS Scheduling, Node Pressure and OOMKilled

Amazon EKS

- Pods may stay in Pending due to insufficient CPU, memory, taints or node affinity.
- Node pressure (CPU, memory, disk) can cause evictions and OOMKilled containers.
- Check node conditions, pod events and resource requests/limits.
- Cluster Autoscaler may take time to add nodes.
- A misconfigured deployment can create more pods than the cluster can schedule.

## 3. Common Container Failure Causes

### Image Issues

- Image not found
- Wrong tag
- Corrupted image
- Vulnerable image
- Large image causing slow start

### IAM Issues

- Missing permissions
- Cannot pull image from ECR
- Cannot access AWS services
- IRSA misconfiguration (in EKS)

### Networking Issues

- Cannot resolve DNS
- Cannot reach external services
- Security group or NetworkPolicy block
- VPC / subnet issue
- Service endpoint issue

### Readiness/Health Issues

- App not ready
- Slow startup time
- Incorrect health check
- Dependency not available (database, cache, external API)
- Crash loop

## 4. ECS and EKS Log Sources

### Amazon ECS

- ECS task events
- ECS service events
- Container logs (CloudWatch Logs)
- ECS service metrics
- ECS Exec for debugging

### Amazon EKS

- kubectl describe
- kubectl get events
- Container logs (CloudWatch Logs)
- CloudWatch Container Insights
- Node logs (via SSM)

## 5. Useful Commands for Troubleshooting

### ECS Commands (AWS CLI)

```
aws ecs describe-tasks \
  --cluster my-cluster \
  --tasks task-id                          # Task details

aws ecs describe-services \
  --cluster my-cluster \
  --services my-service                    # Service status

aws ecs list-tasks \
  --cluster my-cluster \
  --service-name my-service                # Running tasks

aws logs get-log-events \
  --log-group-name /ecs/my-app \
  --log-stream-name stream-name            # Container logs
```

### EKS Commands (kubectl)

```
kubectl get pods -n my-namespace           # Check pod status
kubectl describe pod pod-name \
  -n my-namespace                          # Detailed reason
kubectl get events -n my-namespace \
  --sort-by=".lastTimestamp"               # Recent events
kubectl logs pod-name -n my-namespace      # Application logs
kubectl get nodes                          # Node status
kubectl describe node node-name            # Node conditions
```

### Investigate OOMKilled

```
kubectl describe pod pod-name \
  -n my-namespace | grep -A 5 "OOMKilled"
  # Check OOM reason
kubectl logs pod-name --previous \
  -n my-namespace
  # Logs before crash
kubectl top pods -n my-namespace
  # Resource usage
kubectl top nodes
  # Node resource usage
```

## 6. Architecture: ECS/EKS with Key Dependencies




## 7. Production Scenario

**Scaling activity increases error rates instead of restoring capacity.**

### Possible Causes

- New tasks/pods failing health checks.
- Insufficient cluster capacity (CPU or memory).
- Image pull failures or slow startup time.
- Incorrect readiness or liveness configuration.
- Database or external dependency saturation.
- Autoscaling launching more failing instances.
- Network or security group changes.
- Application bug causing crash loop.

## 8. Troubleshooting Checklist

1. Check service and task/pod status.
2. Review recent deployments and changes.
3. Inspect logs, events and metrics.
4. Check resource usage (CPU, memory, disk).
5. Verify image, IAM, networking and dependencies.
6. Check health check and readiness configuration.
7. Determine if it is application, orchestration or infrastructure related.
8. Apply a targeted fix and verify recovery.

## 9. Senior Engineer Decision

- Distinguish application failures from orchestration (ECS/EKS) and infrastructure failures.
- Use evidence (logs, events, metrics) before making changes.
- Avoid scaling blindly during an incident.
- Fix the real cause, not just the symptom.
- Consider rollback if a new deployment introduced the issue.
- Implement proper health checks, resource requests/limits and observability to prevent recurrence.

# AWS LAMBDA AND SERVERLESS INCIDENTS

## 1. Lambda Invocation Errors, Throttling and Concurrency Exhaustion

AWS Lambda

- Invocation errors (e.g. 429 Too Many Requests, 5xx errors).
- Throttling when requests exceed available concurrency.
- Account or function concurrency limits reached.
- Sustained traffic spikes causing dropped requests.
- Check CloudWatch metrics: Invocations, Errors, Throttles, ConcurrentExecutions.
- Use reserved concurrency to protect critical functions.

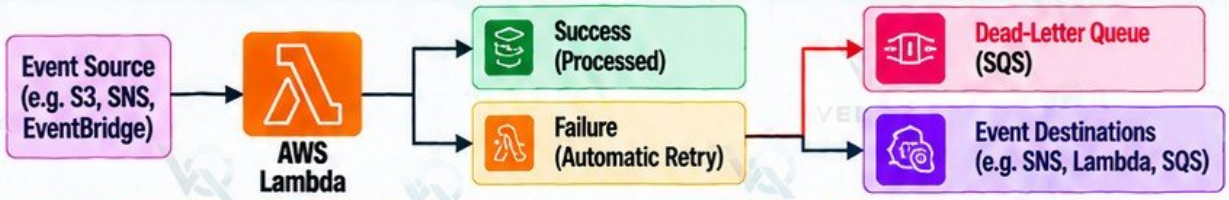
## 2. Cold Starts, Execution Timeouts and Downstream Saturation

- Cold starts increase latency, especially for large packages.
- Execution timeouts when code runs longer than the configured limit.
- High memory usage causing slower execution.
- Downstream services (databases, APIs, queues) become saturated.
- Check duration, memory, init duration, and timeout metrics in CloudWatch.

### Factors That Affect Cold Starts

- Runtime (e.g. Java > Python)
- Package size
- VPC configuration
- Number of concurrent invocations
- Region
- Provisioned Concurrency (reduces cold starts)

## 3. Asynchronous Retries, DLQs and Event Destinations




### Key Points

- Asynchronous invocations are automatically retried (up to 2 more times).
- If all retries fail, the event is sent to a Dead-Letter Queue (DLQ) or an event destination.
- Use DLQ to capture failed events for investigation.
- Configure event destinations for success/failure notifications.
- Monitor age of messages in DLQ and reprocess after fixing the issue.

### Common Retry Issues

- Poison messages causing repeated failures.
- Downstream dependency still unavailable.
- Insufficient concurrency after a traffic spike.
- Function code or configuration errors.
- Messages sit in DLQ and are not reprocessed.

## 4. Amazon API Gateway Integration Failures




### Errors

- 4xx (client error)
- 5xx (integration error)
- Timeout (29 seconds)
- Malformed response
- Permission error (IAM)
- VPC connectivity issue

### Common Causes

- Incorrect integration type (Lambda, HTTP, VPC link).
- Lambda permission not configured (resource policy).
- Mapping template or payload format error.
- Function timeout or error.
- Throttling from Lambda or downstream service.
- VPC link or private integration connectivity issue.

### Integration Types

- Lambda Proxy Integration (recommended)
- Lambda Custom Integration
- HTTP Integration (public or private)
- VPC Link (private resources)
- Mock Integration (testing)

### Key Checks

- Check API Gateway execution logs.
- Verify Lambda resource-based policy.
- Validate request/response mapping.
- Check Lambda function logs in CloudWatch.
- Confirm timeout and payload size.
- Monitor 4xx and 5xx metrics.

## 5. Production Scenario: Retry-Driven Serverless Incident

**A traffic spike creates a retry-driven serverless incident.**

### What Happened?

- Sudden increase in requests to an API.
- Lambda function started returning errors due to downstream database saturation.
- Asynchronous retries increased the load on the function.
- Concurrency limit reached, causing throttling.
- API Gateway began returning 5xx errors.
- Messages accumulated in the DLQ.
- Customer-facing application experienced high error rates.

### Incident Flow

Traffic Spike
↓
Lambda Errors
↓
Automatic Retries
↓
Concurrency Exhaustion
↓
Increased Throttling (429)
↓
API Gateway 5xx Errors
↓
Customer Impact

### How to Resolve

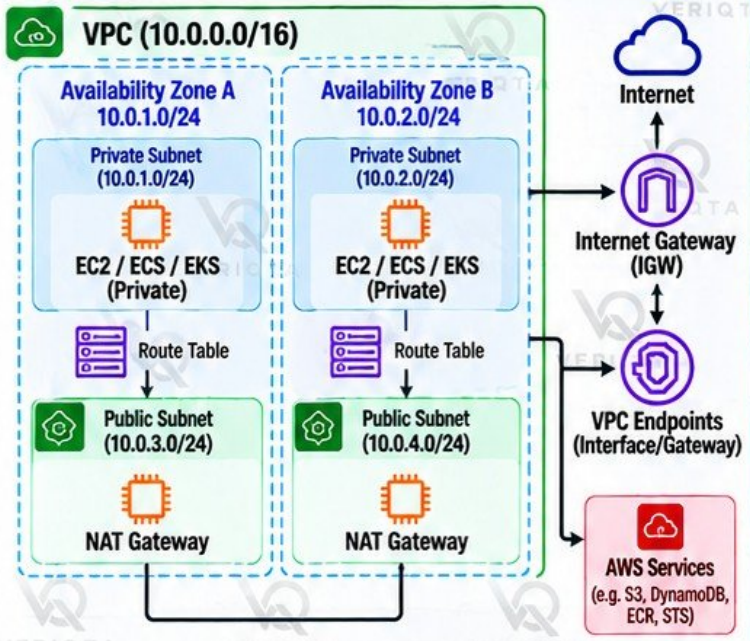
- Identify the initial error and root cause (e.g. database saturation).
- Increase concurrency limit or set reserved concurrency.
- Fix downstream dependency issue.
- Implement backoff and jitter for retries.
- Use DLQ to stop retry loops for poison messages.
- Consider circuit breaker or rate limiting.
- Monitor and verify recovery in CloudWatch.

### Key Takeaways

- Monitor downstream dependencies, not just Lambda.
- Set appropriate concurrency limits and alarms.
- Use retries carefully to avoid amplification.
- Implement DLQs and event destinations.
- Design for graceful failure and backpressure.
- Test how your serverless application behaves during traffic spikes.

# VPC CONNECTIVITY AND ROUTING FAILURES

## 1. VPC Networking Components




## 2. Route Tables, Security Groups and NACLs

### Route Table (Subnet Level)

| Destination | Target |
|---|---|
| 10.0.0.0/16 | local |
| 0.0.0.0/0 | nat-xxxxxxexx |

**Key Points**

- Controls where traffic goes from a subnet.
- Each subnet must have a route to reach external networks (via NAT or IGW).
- Incorrect routes cause connectivity failures.

### Security Groups (Instance/ENI Level)

| Type | Protocol/Port | Source/Destination |
|---|---|---|
| Inbound | TCP 80 | 0.0.0.0/0 (or ALB SG) |
| Inbound | TCP 22 | Bastion SG |
| Outbound | All | 0.0.0.0/0 |

**Key Points**

- Stateful firewall.
- Allow only required traffic.
- Return traffic is automatically allowed.

### Network ACLs (Subnet Level)

| Type | Protocol/Port | Source | Source |
|---|---|---|---|
| Inbound | TCP 80 | 0.0.0.0/0 | Allow |
| Inbound | All | 0.0.0.0/0 | Deny |
| Outbound | All | 0.0.0.0/0 | Allow |

**Key Points**

- Stateless firewall (must allow both inbound and outbound).
- Evaluated in order by rule number.
- Explicit allow and deny rules required.

## 3. NAT Gateway, Internet Gateway and VPC Endpoints

### NAT Gateway

- Allows private subnet resources to access the internet.
- Used for outbound traffic (e.g. package updates).
- Common failures:
  - No route to NAT
  - NAT gateway exhausted or deleted
  - Elastic IP issues
  - High connection/port usage (ephemeral ports)

### Internet Gateway (IGW)

- Attaches to the VPC.
- Allows public subnets to access the internet.
- Used for internet-facing resources (e.g. ALB).
- Common failures:
  - Not attached to VPC
  - Missing route in public subnet
  - Security group or NACL blocking traffic

### VPC Endpoints

- Private connectivity to AWS services (e.g. S3, DynamoDB, ECR).
- Types: Gateway (S3, DynamoDB) and Interface (most services).
- Common failures:
  - Endpoint policy blocking access
  - Missing DNS resolution
  - Not associated with subnet
  - Private DNS disabled
  - Security group on endpoint blocking traffic

## 4. VPC Flow Logs and Reachability Analyzer

### VPC Flow Logs

- Capture information about IP traffic going to and from network interfaces.
- Helps identify rejected traffic, port issues and security group/NACL blocks.
- View in CloudWatch Logs or S3.
- Use filters to find REJECT entries.

```
2 123456789012 eni-0abcde12345 10.0.1.10 52.95.110.1
443 6 20 12345 67890 REJECT -
```

### Reachability Analyzer

- Tests network reachability between resources (e.g. EC2 to RDS, EC2 to internet).
- Identifies misconfigurations in route tables, security groups, NACLs and network firewalls.
- Shows the exact path and the blocking component.

## 5. Common VPC Connectivity Issues

| Issue | Symptoms | Possible Causes |
|---|---|---|
| Cannot reach internet from private subnet | Connection timeouts, package update failures | • Missing NAT gateway or route<br>• NAT gateway deleted or exhausted<br>• Security group/NACL block<br>• DNS resolution issue |
| Public subnet cannot reach internet | No outbound connectivity | • IGW not attached<br>• Missing route (0.0.0.0/0 to IGW)<br>• NACL blocking traffic |
| Cannot reach AWS service (e.g. S3) | Timeouts or access denied | • Missing/broken VPC endpoint<br>• Endpoint policy blocking<br>• DNS/Private DNS issue |
| Asymmetric routing | Intermittent failures, connection drops | • Different route for return traffic<br>• Multi-homed setup or peering issue |
| Ephemeral port exhaustion | Random connection failures | • High outbound connections (NAT)<br>• Insufficient NAT gateway ports |

## 6. Production Scenario

**Private workloads lose access to an external dependency.**

**Scenario**

- EC2 instances in private subnets cannot reach an external API.
- Application logs show connection timeouts.
- VPC Flow Logs show REJECT entries.
- NAT gateway was healthy but hit connection limit due to a traffic spike.
- No VPC endpoint available for the service.

**Resolution**

- Confirm from Flow Logs and Reachability Analyzer.
- Increase NAT gateway capacity or add a second NAT per AZ.
- Use VPC endpoint if available.
- Implement connection reuse and retry logic.

## 7. Key Takeaways

- Verify route tables, security groups and NACLs.
- Use VPC Flow Logs to identify rejected traffic.
- Use Reachability Analyzer to test connectivity.
- Understand NAT gateway limits and ephemeral ports.
- Check DNS resolution for VPC endpoints.
- Investigate asymmetric routing in complex networks.
- Monitor connection counts and timeouts during traffic spikes.

# ROUTE 53, DNS, AND TLS INCIDENTS

## 1. DNS Resolution and Resolver Behavior




### Key Points

- Client queries a recursive DNS resolver.
- Resolver checks cache first (TTL).
- If not cached, it queries Amazon Route 53 (authoritative).
- Route 53 returns the DNS record (A, AAAA, CNAME, etc.).
- Result is cached based on TTL.
- Issues can occur at any step (client, resolver, cache, or authoritative).

## 2. Route 53 Routing Policies, Health Checks and Failover

| Routing Policy | Use Case |
|---|---|
| Simple | Single resource (no failover) |
| Weighted | Load distribution, canary releases |
| Latency-based | Route to lowest latency Region |
| Geolocation | Route based on user location |
| Geoproximity | Route based on geographic location (with bias) |
| Failover | Primary/secondary with health checks |
| Multivalue Answer | Return multiple healthy resources |

### Health Checks

- Monitors endpoint health (HTTP, HTTPS, TCP, etc.).
- Used with failover routing.
- Unhealthy resources are automatically removed from DNS responses.

### Failover Example

Primary (Healthy)
↓
Health Check Fails
↓
Secondary (Used)

## 3. DNS TTL, Caching and Propagation

### TTL (Time To Live)

- Determines how long a DNS record is cached.
- Lower TTL = faster changes, but more DNS queries.
- Higher TTL = longer caching, slower propagation, less load.

### Caching Layers

- Client OS cache (e.g. Windows, macOS, Linux)
- Browser cache
- ISP or corporate resolver cache
- Recursive resolver cache (e.g. 8.8.8.8)
- Each layer respects TTL.

### Propagation Implications

- DNS changes are not instant worldwide.
- Existing cached records remain until TTL expires.
- Use low TTL before planned changes.
- Allow time for global propagation during incident recovery.

## 4. AWS Certificate Manager and TLS Failures

**AWS Certificate Manager (ACM)**

### Certificate Lifecycle

- Request certificate (public or private).
- Domain validation (DNS or email).
- Certificate issued.
- Automatic renewal (for public certs).
- Deployed to services (ALB, CloudFront, API Gateway, etc.).
- Monitor expiration and renewal status.

### Common TLS Failures

- Certificate expired or not yet valid.
- Hostname mismatch.
- Untrusted certificate.
- Incomplete certificate chain.
- Unsupported TLS version or cipher.
- Misconfigured certificate on ALB, CloudFront, or API Gateway.

### Troubleshooting Commands

- `openssl s_client -connect example.com:443 -servername example.com`
- `curl -I https://example.com`
- `nslookup example.com`
- `dig example.com`

## 5. Common DNS and TLS Incident Scenarios

| Scenario | Symptoms | Possible Causes |
|---|---|---|
| DNS resolution failure | Cannot resolve domain name | • Incorrect DNS record<br>• Route 53 misconfiguration<br>• Resolver or ISP issue<br>• DNS record deleted |
| Intermittent resolution | Works sometimes, fails sometimes | • Low TTL with caching<br>• Multiple DNS records (routing policy)<br>• Resolver cache inconsistency |
| Incorrect routing | Users routed to wrong Region or environment | • Wrong routing policy configuration<br>• Health check misconfiguration<br>• Stale DNS cache |
| TLS handshake failure | SSL/TLS error in browser or application | • Expired or invalid certificate<br>• Hostname mismatch<br>• Unsupported TLS version or cipher<br>• Incomplete certificate chain |
| Certificate renewal issue | Service outage after certificate expiration | • ACM renewal failed<br>• DNS validation issue<br>• Certificate not deployed to service<br>• Manual certificate not renewed |

## 6. Production Scenario: DNS and TLS Incident

**Users cannot access the application after a DNS or certificate change.**

User reports cannot access site
↓
Check DNS resolution (nslookup / dig)
↓
Verify Route 53 records and health checks
↓
Check certificate status in ACM
↓
Test TLS handshake (openssl / curl)
↓
Identify root cause (DNS, TLS, network or application)

### Potential Causes

- Incorrect or deleted Route 53 record.
- Health check marked healthy incorrectly.
- Stale DNS cache still pointing to old endpoint.
- Certificate expired or misconfigured.
- Certificate not deployed to the service.
- TLS version or cipher incompatibility.
- Network or application issue misidentified as a DNS/TLS problem.

## 7. Senior Engineer Decision

- Differentiate DNS, TLS, network, and application-layer symptoms.
- Validate DNS records, health checks, and TTL before making changes.
- Check certificate status, validity, and deployment.
- Use evidence from multiple sources (DNS tools, SSL tools, logs, monitoring).
- Communicate impact and expected resolution time clearly.
- Plan changes carefully to avoid extended outages due to caching or propagation.

### Key Takeaways

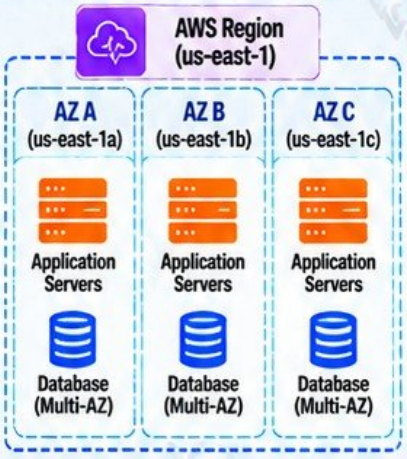
- Always verify where the failure is occurring (DNS, TLS, network or application).
- Check DNS resolution from multiple locations.
- Understand TTL and caching behavior.
- Monitor Route 53 health checks and ACM certificate expiration.
- Use proper diagnostics to isolate the true root cause.
- Avoid making changes without understanding the impact on global users.

# MULTI-AZ AND REGIONAL OUTAGE RESPONSE

## 1. Availability Zone vs Regional Failure Domains

### Availability Zone (AZ)

- Isolated data center with independent power, cooling and networking.
- Designed to protect against single data center failures.
- Low-latency connectivity between AZs in the same Region.
- Use Multi-AZ deployments to remain available if one AZ fails.




### AWS Region

- Geographic location containing multiple AZs (e.g. us-east-1).
- Isolated from other Regions.
- Provides higher resilience against large-scale events.
- Use multi-Region architecture to protect against Regional outages.
- Some AWS services are Regional (e.g. DynamoDB, RDS, S3) and must be designed for Regional failure.

## 2. AWS Health Signals and Dependency Assessment

**AWS Health**

- Provides real-time information about AWS service issues and planned events.
- View Regional, AZ and service-specific health events.
- Subscribe to notifications (SNS, EventBridge).
- Check for dependent service impacts (e.g. EC2, RDS, ELB, Route 53).
- Correlate AWS Health with CloudWatch metrics, logs and external monitoring.

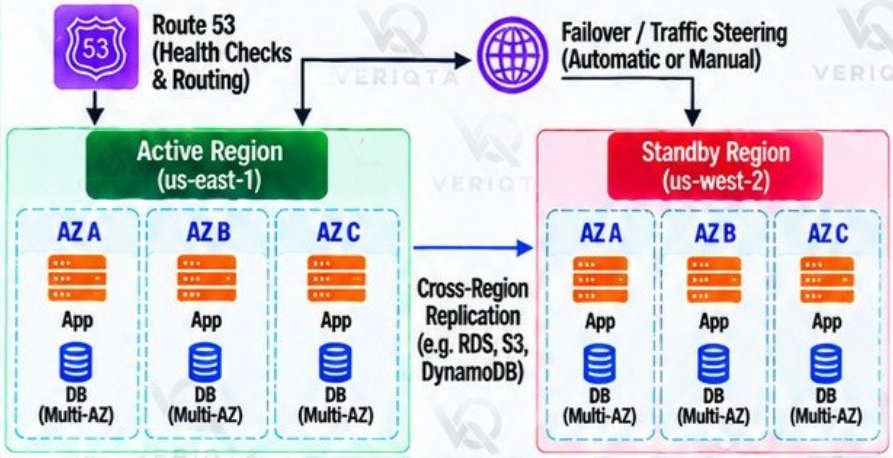
### Key Signals

- Provice disruptions
- Increased error rates
- Availability Zone impairments
- Planned maintenance events
- Dependency chain analysis

### Tools

- AWS Health Dashboard
- AWS Health API
- Amazon EventBridge (health events)
- Amazon SNS notifications

## 3. Multi-AZ Failover, Regional Evacuation and Traffic Steering




### Traffic Steering Options

- Amazon Route 53 health checks and failover routing.
- Latency-based or geolocation routing.
- Active-active (both Regions serve traffic).
- Active-passive (failover to standby Region).
- Use global accelerators or CloudFront for global traffic management.

### Regional Evacuation Considerations

- Automate failover where possible.
- Validate data replication and service dependencies.
- Update DNS/traffic routing.
- Ensure capacity in target Region.
- Test evacuation procedures regularly.

## 4. Recovery Time Objective (RTO) and Recovery Point Objective (RPO)

### Recovery Time Objective (RTO)

- Maximum acceptable time to restore service after an outage.
- Determines how quickly you must fail over and recover.
- Influences architecture, automation and operational procedures.

### Recovery Point Objective (RPO)

- Maximum acceptable amount of data loss measured in time.
- Determines how often data must be replicated or backed up.
- Influences database replication, backup strategy and data protection services.

### Example RTO and RPO by Workload

| Workload Type | Typical RTO | Typical RPO |
|---|---|---|
| Customer-facing web app | Minutes | Minutes |
| E-commerce platform | Minutes | Seconds |
| Internal business app | Hours | Hours |
| Analytics / batch processing | Hours | Hours |
| Mission-critical financial system | Minutes | Seconds |

## 5. Architecture Trade-off: Active-Active vs Active-Passive

### Active-Active (Multi-Region)

- Both Regions actively serve traffic.
- Higher cost.
- Complex architecture and data synchronization.
- Lower RTO and RPO.
- Suitable for critical, global applications.

### Active-Passive (Primary-Secondary)

- One Region active, one Region on standby.
- Lower cost.
- Simpler architecture.
- Higher RTO (depends on failover time).
- Suitable for many business applications with moderate recovery requirements.

## 6. Production Scenario

**A regional dependency disrupts an otherwise multi-AZ application.**

### What Happened?

- Application is deployed across multiple AZs in us-east-1.
- A dependent service (e.g. DynamoDB, API, or third-party service) in the Region experiences an outage.
- All AZs in the Region are affected.
- Application becomes unavailable despite multi-AZ deployment.

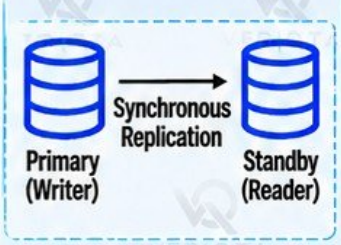
## 7. Key Takeaways

- Multi-AZ protects against AZ failures, not Regional outages.
- Monitor AWS Health for Regional service issues.
- Understand and map dependencies across services and Regions.
- Design for appropriate RTO and RPO based on business requirements.
- Choose active-active or active-passive based on cost, complexity and recovery goals.
- Test failover and regional evacuation regularly.

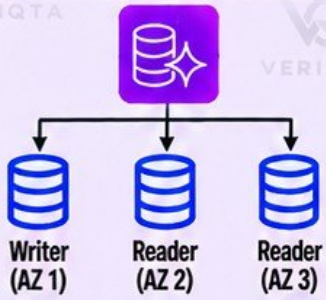
# RDS AND AURORA DATABASE INCIDENT RESPONSE

## 1. RDS and Aurora Architecture Overview

### Amazon RDS (Multi-AZ)




### Amazon Aurora (Multi-AZ / Multi-Region)




## 2. Common Database Incident Types

| Incident Type | Description | Symptoms |
|---|---|---|
| Connection Exhaustion | Too many concurrent connections. | New connections fail, application errors. |
| Transaction Contention | Long-running transactions blocking others. | Increased latency, lock waits. |
| Deadlocks | Conflicting transactions. | Deadlock errors in logs, failed transactions. |
| Slow Queries | Inefficient queries or missing indexes. | High query latency, increased CPU/IO. |
| Replica Lag | Read replica behind primary. | Replication lag metric increases. |
| Storage Pressure | High storage usage or IOPS limits. | Write latency, storage full errors. |

## 3. Monitoring and Diagnostic Tools

### RDS Events

- View DB instance events in the RDS console.
- Identifies failover, maintenance, and error events.
- Helps understand timeline and root cause.

### Performance Insights (Database Insights)

- Visualize database load and performance.
- Identify top SQL queries, wait events, and bottlenecks.
- Compare performance before and during the incident.

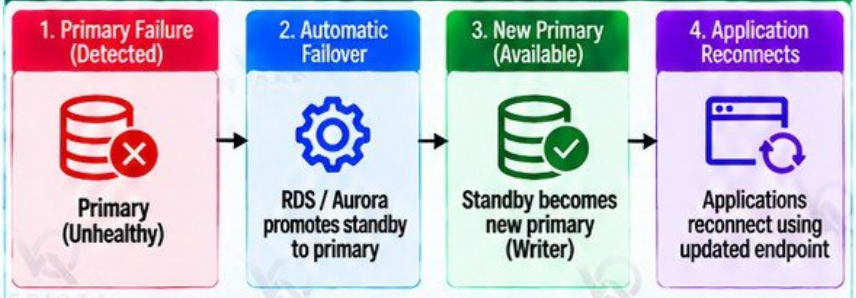
### Amazon CloudWatch

- Monitor CPU, memory, connections, storage, read/write latency.
- Key metrics: DatabaseConnections, CPUUtilization, FreeStorageSpace, Read/Write IOPS, ReplicaLag.
- Set alarms for abnormal behavior.

### Database Logs

- Enable and analyze database logs (error logs, slow query logs, audit logs).
- Look for connection errors, deadlocks, slow queries, and replication issues.

## 4. Multi-AZ Failover and Connection Recovery




## 5. Key Metrics to Investigate

| Metric | What It Indicates |
|---|---|
| DatabaseConnections | High or near limit = connection exhaustion |
| CPUUtilization | High CPU = query or load issue |
| FreeableMemory | Low memory = performance degradation |
| FreeStorageSpace | Low storage = write failures |
| ReadLatency / WriteLatency | High latency = performance issue |
| ReadIOPS / WriteIOPS | High I/O = possible query or load issue |
| ReplicaLag | High lag = replica behind primary |
| DiskQueueDepth | High queue depth = storage pressure |

## 6. Troubleshooting Steps

1. Check RDS events for failover, errors, or maintenance.
2. Review CloudWatch metrics for CPU, connections, storage, and latency.
3. Use Performance Insights to identify top queries and wait events.
4. Analyze database logs for errors, deadlocks, or slow queries.
5. Check replica lag and replication health (for read replicas).
6. Verify application connection strings and DNS resolution.
7. Confirm security groups and network connectivity.
8. Check for storage or IOPS limits.

## 7. Production Scenario

**Database failover completes, but application errors persist.**

### What Happened?

- Primary RDS instance failed and Multi-AZ failover completed successfully.
- The database is available and accepting connections.
- Applications continue to see errors (e.g. connection timeouts, authentication failures, or high latency).

### Possible Causes

- Applications still using old DNS endpoint cached in local DNS or connection pool.
- Stale connections in application layer.
- Connection pool not refreshing after failover.
- Security group or network issue.
- Application configuration pointing to the old primary.
- High load after failover causing slow responses.

## 8. Senior Engineer Decision

- Confirm database health before assuming application issue.
- Validate application connection handling and DNS caching.
- Check for connection pool limits and retry logic.
- Distinguish between database, network, and application-layer symptoms.
- Communicate clear impact and resolution steps to stakeholders.

## 9. Key Takeaways

- Monitor connections, query performance, and storage.
- Use RDS events, Performance Insights, and CloudWatch.
- Understand Multi-AZ failover and recovery behavior.
- Check application configuration and connection handling.
- Database availability does not always mean application recovery.
- Investigate all layers: database, network, and application.

# S3, EBS, AND EFS STORAGE INCIDENTS

## 1. Storage Services Overview

### Amazon S3



- Object storage for any amount of data.
- Highly durable and scalable.
- Used for backups, logs, static content, data lakes, and more.

### Amazon EBS



- Block storage for EC2 instances.
- Provides persistent and low-latency storage.
- Optimized for performance (IOPS/throughput).

### Amazon EFS



- Managed file storage (NFS).
- Shared access across multiple EC2 instances.
- Scales automatically.
- Used for applications requiring shared file systems.

## 2. Common Storage Incident Types

| Service | Incident Type | Description | Symptoms |
|---|---|---|---|
| S3 | Access Failure | Permission or policy misconfiguration. | 403 errors, AccessDenied. |
| | Throttling | High request rate exceeds limits. | 429 errors, slow responses. |
| | Data Not Found | Object deleted or wrong key. | 404 errors, missing objects. |
| EBS | Performance Issue | Low throughput or high latency. | Slow I/O, high read/write latency. |
| | IOPS Exhaustion | Exceeded provisioned IOPS. | Increased latency, I/O errors. |
| | Volume Health | Underlying hardware issue. | Volume in impaired state, I/O errors. |
| EFS | Mount Failure | Security group, NACL or DNS issue. | Cannot mount, connection timeouts. |
| | Throughput Issue | Burst credits exhausted or mode limits. | Slow file access, high latency. |
| | Client Issue | Application or OS-level problem. | Stale file handles, I/O errors. |

## 3. S3 Access Failures, Policy Errors, and Throttling

### Common Causes

- Incorrect IAM permissions (user, role, or bucket policy).
- Block Public Access settings.
- Wrong region or endpoint.
- VPC endpoint policy restrictions.
- Request throttling (too many requests).
- Object ACL or bucket policy conflicts.

### S3 Access Flow




## 4. EBS Performance, IOPS, Latency, and Volume Health

### Key Metrics (CloudWatch)

- VolumeReadOps / VolumeWriteOps
- VolumeReadBytes / VolumeWriteBytes
- VolumeQueueLength
- VolumeThroughputPercentage
- VolumeConsumedReadWriteOps
- VolumeIdleTime

### Common Issues

- Insufficient IOPS for workload.
- Throughput limits reached.
- High latency due to burst credit exhaustion (gp2).
- Incorrect volume type or size.
- Volume in impaired state.
- File system corruption.

### Volume Types and Use Cases

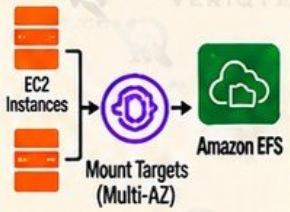
| Type | Best For |
|---|---|
| gp3 | General purpose (balanced) |
| io2 | High performance, mission critical |
| st1 | Throughput intensive (sequential) |
| sc1 | Cold data (low cost) |

## 5. EFS Throughput Modes and Mount Issues

### Throughput Modes

| | |
|---|---|
| Bursting | Scales up to a baseline and uses credits. |
| Provisioned | Set a fixed throughput regardless of usage. |
| Elastic | Automatically scales with demand. |

### EFS Architecture




### Common Mount Issues

- Incorrect security group or NACL. DNS resolution problems.
- Mount target not available in the AZ.
- IAM permissions (for EFS IAM auth).
- Stale file handles or client-side issues.

## 6. Versioning, Snapshots, Restore Procedures, and Data Integrity

### S3 Versioning

- Keeps multiple versions of an object.
- Helps recover from accidental deletion or overwrite.
- Enable versioning at the bucket level.
- Use lifecycle policies to manage old versions.

### EBS Snapshots

- Point-in-time backup of a volume.
- Stored in S3 and persistent.
- Can be used to create new volumes.
- Supports cross-region and cross-account copy.

### EFS Backup and Restore

- Use AWS Backup for automated backups.
- Restore to the same or a different file system.
- Maintains file system metadata and permissions.
- Validate data integrity after restore.

## 7. Incident Classification

| Type | Description | Examples |
|---|---|---|
| Access | Cannot access the storage resource. | S3 AccessDenied, EFS mount failure. |
| Capacity | Storage limits reached. | EBS out of space, EFS throughput limit. |
| Performance | Slow or degraded performance. | High latency, low IOPS, throttling. |
| Data Loss | Data is missing, corrupted or deleted. | Deleted S3 objects, corrupted file system. |

## 8. Senior Engineer Decision



- Determine if the incident is access, capacity, performance, or data loss.
- Check recent configuration changes and IAM policies.
- Validate metrics, logs, and storage health status.
- Consider impact on dependent services and customer experience.
- Choose the right recovery procedure (policy fix, increase capacity, restore from snapshot, etc.).

## 9. Production Scenario

Database snapshots stored in S3 become inaccessible due to an incorrect bucket policy, causing backup failures and increasing recovery risk.

### What Happened?

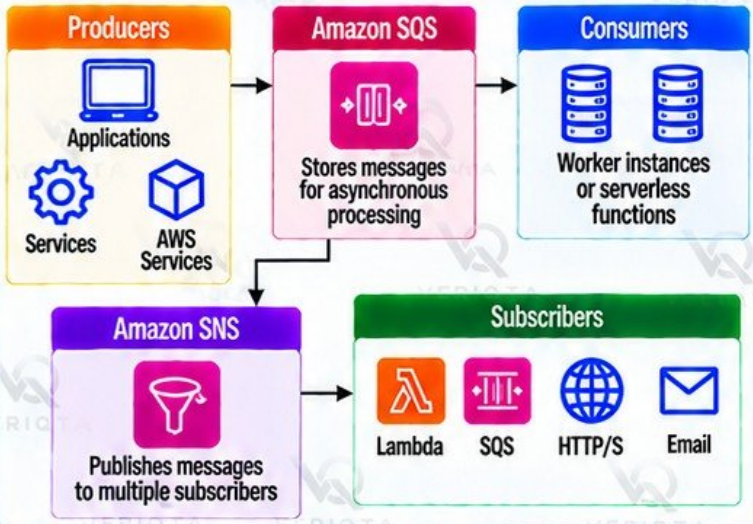
- S3 bucket policy was changed.
- Backup jobs started failing with 403 errors.
- No new backups were created for 24 hours.
- Recovery procedures had to use older backups.

## 10. Key Takeaways

- Understand differences between S3, EBS, and EFS.
- Monitor access, capacity, and performance metrics.
- Enable versioning and automated backups.
- Test restore procedures regularly.
- Investigate storage incidents systematically and communicate impact clearly.

# SQS, SNS, AND EVENT-DRIVEN FAILURE RECOVERY

## 1. Event-Driven Architecture Overview




## 2. Common Incident Types

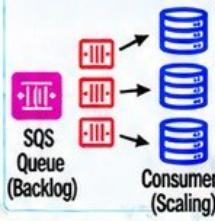
| Service | Incident Type | Description | Symptoms |
|---|---|---|---|
| SQS | Queue Backlog | Messages build up faster than consumers can process. | High Approximate Number of Messages, increasing message age. |
| | Consumer Saturation | Consumers cannot keep up with message volume. | High CPU/memory, scaling lag, increased processing time. |
| | Visibility Timeout Issues | Messages become visible again before processing completes. | Duplicate processing, multiple consumers processing same message. |
| | Poison Messages | Messages always fail processing. | Repeated failures, high receive count, messages in DLQ. |
| SNS | Delivery Failures | SNS cannot deliver message to a subscriber. | Delivery retries, failed deliveries in metrics, error logs. |
| | Downstream Outages | Subscriber endpoint is unavailable. | Increased delivery failures, throttling, timeouts. |

## 3. SQS Queue Backlog, Message Age, and Consumer Saturation

### Key Metrics (CloudWatch)

- ApproximateNumberOfMessagesVisible
- ApproximateNumberOfMessagesNotVisible
- ApproximateAgeOfOldestMessage
- NumberOfMessagesSent
- NumberOfMessagesReceived
- NumberOfMessagesDeleted

### Backlog Example

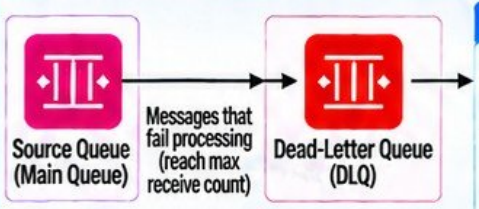



If consumers cannot keep up, message age increases and backlog grows.

## 4. Visibility Timeouts and Duplicate Processing




## 5. Dead-Letter Queues (DLQ) and Safe Redrive

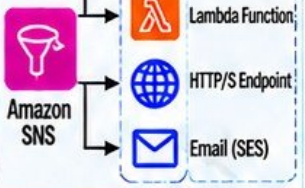



### Investigation Steps

- Analyze failed messages.
- Identify root cause.
- Fix underlying issue.
- Test with a small batch.
- Redrive messages safely back to the source queue.

## 6. SNS Delivery Failures and Downstream Outages

### SNS to Multiple Subscribers




### Common Causes

- Invalid endpoint or permissions.
- Subscriber service outage.
- Throttling or rate limits.
- Network or VPC connectivity issues.
- Message size limits exceeded.
- Misconfigured retry policy.

### Monitoring and Logs

- SNS delivery metrics in CloudWatch.
- Delivery status logs (for HTTP/S, email).
- Subscription health checks.

## 7. Production Scenario: Recovering Consumers Overwhelm the Database




### What Happened?

- Consumers recover and process backlog rapidly.
- High concurrent database requests.
- Database experiences high CPU, connection exhaustion, or throttling.
- Application errors increase despite queue backlog reducing.

## 8. Troubleshooting Steps

1. Check queue metrics and message age.
2. Investigate consumer health and scaling.
3. Analyze failed messages and DLQ.
4. Verify visibility timeout and retry behavior.
5. Check SNS delivery status and subscriber health.
6. Review downstream service health (e.g. database).
7. Throttle or control consumer scale if needed.
8. Redrive messages after fixing the root cause.

## 9. Senior Engineer Decision

- Determine if the issue is message volume, processing speed, consumer scaling, or downstream dependency.
- Balance retry behavior, visibility timeout, and DLQ usage.
- Implement rate limiting or controlled scaling to protect downstream services.
- Communicate impact and resolution steps to stakeholders.

## 10. Key Takeaways

- Monitor queue depth, message age, and consumer health.
- Set appropriate visibility timeouts.
- Use DLQs and redrive safely.
- Monitor SNS delivery failures and subscriber endpoints.
- Protect downstream services from sudden load after recovery.
- Understand and tune retry, throttling, and scaling behavior.

# IAM AND ACCESS DENIED INCIDENT INVESTIGATION

## 1. IAM Components and How Permissions Work

### Identity Policies
(Attached to a user, group or role)

Define permissions for the identity.

- Allow specific actions on AWS services.
- Can be inline or managed policies.

### Resource Policies
(Attached to a resource)

Define permissions for a resource.

- Used by services like S3, Lambda, SNS, etc.
- Can allow or deny specific principals.

### Explicit Deny
(Overrides Allow)

An explicit deny always overrides any allow.

- Can be in identity policy, resource policy, SCP, permission boundary or session policy.

### Effective Permissions

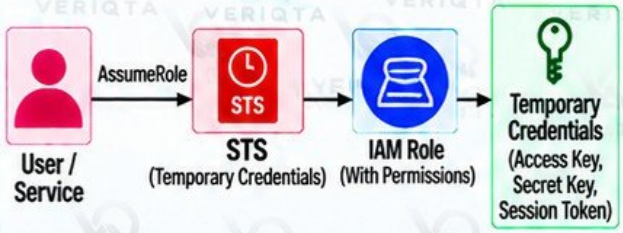
The final permission is the result of evaluating all policies.

- Explicit deny overrides allow.
- All applicable policies must allow the action.

## 2. Advanced IAM Policy Controls

| Policy Type | Purpose | Key Points |
|---|---|---|
| Service Control Policies (SCPs) | Set maximum permissions in AWS Organizations. | Cannot grant permissions. Only limits what can be done, even if identity policy allows it. |
| Permission Boundaries | Set the maximum permissions for an IAM identity (user or role). | The identity's effective permissions cannot exceed the boundary. |
| Session Policies | Further restrict permissions for a session (e.g. via STS). | Can only reduce permissions, not increase them. |

## 3. STS Sessions, Role Assumption, and Credential Expiration




### Key Considerations

- STS provides temporary credentials for assumed roles.
- Credentials have an expiration time (e.g. 15 minutes to 12 hours).
- Applications must refresh credentials before they expire.
- Expired credentials result in AccessDenied errors.

## 4. Common Access Denied Causes

### Common Causes

- Missing permissions in identity policy.
- Explicit deny in any policy (identity, resource, SCP, permission boundary, or session policy).
- SCP blocking the action.
- Permission boundary limiting the identity.
- Incorrect role assumption or trust policy.
- Expired or invalid temporary credentials.
- Resource policy does not allow the principal.
- Incorrect AWS region or resource ARN.
- Service-specific conditions (e.g. VPC endpoint, SourceIp, aws:PrincipalOrgID).
- Cross-account access misconfiguration.

## 5. Investigating Access Denied with CloudTrail

### 1. Find the Event in CloudTrail

Search for the failed action in CloudTrail.

- Look for eventName, errorCode = AccessDenied, and errorMessage.
- Identify the principal, source IP, and request parameters.

### 2. Analyze the Details

```
{
  "eventName": "GetObject",
  "errorCode": "AccessDenied",
  "errorMessage": "User is not authorized to perform s3:GetObject on this resource",
  "userIdentity": { ... },
  "requestParameters": { ... }
}
```

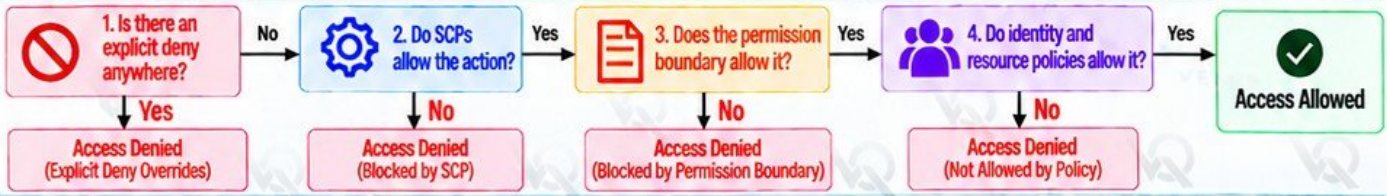
### 3. Identify the Source

- Which user, role, or service made the request?
- Which resource was accessed?
- What policy or condition might be denying it?
- Is the request from the expected region and account?

### 4. Use IAM Policy Simulation

- Use the IAM Policy Simulator to test permissions.
- Check which policies allow or deny the action.
- Test with the exact action, resource and conditions to identify the issue.

## 6. Policy Evaluation Flow (Simplified)




## 7. Production Scenario: Deployment Succeeds, Application Loses Access

### What Happened?

- A new version of the application is deployed successfully.
- After deployment, the application starts getting AccessDenied errors when calling AWS services.
- The deployment changed the IAM role or policy.
- The issue is caused by a missing permission, an explicit deny, or an updated SCP/permission boundary.

### Investigation Steps

1. Check application logs for AccessDenied errors.
2. Find the failed events in CloudTrail.
3. Compare the old and new IAM role/policy.
4. Use the IAM Policy Simulator to test permissions.
5. Verify SCPs and permission boundaries.
6. Confirm the correct role is being assumed.
7. Fix the policy and redeploy or refresh credentials.

## 8. Key Takeaways

- Always check for explicit denies.
- Consider all applicable policies (identity, resource, SCP, permission boundary, session policy).
- Use CloudTrail to find the exact cause.
- Validate permissions with IAM Policy Simulator.
- Monitor credential expiration for temporary credentials.
- Document and test IAM changes before production deployment.

# COMPROMISED CREDENTIALS AND AWS SECURITY INCIDENTS

## 1. Common Attack Vectors and Credential Exposure

### Exposed Credentials

- Hardcoded credentials in code or configuration.
- Credentials in public repositories.
- Leaked access keys.
- Stolen developer workstations.
- Phishing and social engineering.

### Unauthorized Access

- Use of valid credentials from unknown IP addresses.
- Access from unusual geographic locations.
- Compromised CI/CD pipelines.
- Stolen IAM roles or instance profiles.

### Privilege Escalation

- Assuming higher privileged roles.
- Exploiting overly permissive policies.
- Lateral movement across services.
- Creating new users, roles, or access keys.

### Common Indicators

- Unusual API activity.
- Access from rare locations.
- Large data downloads or deletions.
- Unexpected resource changes.
- New IAM users or policies created.

## 2. GuardDuty Findings and Suspicious Activity

### Common GuardDuty Findings

- UnauthorizedAccess:IAMUser
- UnauthorizedAccess:AssumedRole
- Recon:EC2/PortProbe
- CryptoCurrency:EC2/BitcoinTool
- Trojan:EC2/BlackHoleTraffic
- Persistence:EC2/StealthLaunch
- Policy:IAMUser/RootCredentialUsage

### Suspicious API Activity

- CreateUser / DeleteUser
- CreateAccessKey
- AssumeRole
- AttachUserPolicy
- CreatePolicyVersion
- ModifySecurityGroup
- DisableCloudTrail
- Access from unusual IPs or regions

## 3. Investigation with CloudTrail and AWS Logs

### AWS CloudTrail (API Activity Logs)

- Identify the timeline of malicious activity.
- Find the source IP, user, or role.
- Review API calls and changes.
- Check for credential abuse and privilege escalation.

- CloudTrail Log Events
- CloudWatch Logs (Additional Context)
- VPC Flow Logs (Network Activity)
- S3 Access Logs (Data Access)

### Investigation Questions

- What identity was used?
- What actions were performed?
- When did the activity start and from where?
- Which resources were accessed or modified?
- Was data accessed or exfiltrated?
- What was the full scope of the compromise?

## 4. Containment Steps

1. Revoke compromised access keys immediately.
2. Disable or remove compromised IAM users.
3. Rotate credentials and invalidate active sessions.
4. Detach or restrict affected IAM roles and policies.
5. Isolate affected workloads (e.g. revoke instance profile, remove network access).
6. Review and restrict security groups and network access.
7. Enable additional monitoring and GuardDuty if not already.
8. Confirm the attacker no longer has access.

## 5. Evidence Preservation and Forensics

### Preserve Logs

- Retain CloudTrail logs.
- Preserve GuardDuty findings.
- Keep VPC Flow Logs, S3 access logs, and CloudWatch logs.
- Do not delete or modify logs during containment.

### Capture Key Evidence

- Record timestamps, user/role, source IPs, and API calls.
- Export relevant log events.
- Capture affected resources and configuration changes.
- Document all actions taken during incident response.

### Analyze the Timeline

- Determine initial entry point.
- Identify lateral movement.
- Understand the full scope of actions.
- Look for data access or exfiltration.

### Tools to Use

- AWS CloudTrail
- Amazon GuardDuty
- Amazon CloudWatch
- AWS Config (resource changes)
- VPC Flow Logs
- IAM Access Analyzer
- AWS CLI and CloudTrail Insights

## 6. Production Scenario

### What Happened?

- A developer's access key is accidentally committed to a public repository.
- An attacker uses the key to assume an IAM role with elevated permissions.
- The attacker creates resources, accesses sensitive data, and attempts to disable CloudTrail.
- The security team detects unusual activity through GuardDuty and CloudTrail.
- Containment is performed while preserving evidence for forensic analysis.

## 7. Senior Engineer Decision

- Contain the attacker's access without destroying forensic evidence.
- Prioritize protection of critical data and workloads.
- Preserve logs and maintain the incident timeline.
- Determine the scope of compromise and potential data exposure.
- Decide on credential rotation, policy updates, and long-term security improvements.
- Communicate with stakeholders and coordinate further investigation.

## 8. Key Takeaways

- Monitor GuardDuty findings and unusual API activity.
- Protect credentials and use least privilege.
- Detect and respond quickly to suspicious activity.
- Preserve evidence for investigation.
- Contain the incident without deleting logs.
- Review and strengthen IAM policies, access controls, and monitoring.

# AWS INCIDENT INVESTIGATION WITH LOGS AND TRACES

## 1. Observability Components for Incident Investigation

### Amazon CloudWatch Logs

- Collects and stores log data from AWS services and applications.
- Use Logs Insights for fast, structured queries.
- Helps identify errors, latency, and unusual patterns.

### AWS CloudTrail

- Records API calls and account activity.
- Management events (show control plane activity).
- Data events (show resource level activity such as S3 object access).
- Essential for security and operational investigations.

### AWS X-Ray

- Provides distributed tracing for microservices and serverless applications.
- Visualizes request flow across services.
- Helps find latency bottlenecks and errors.
- Integrates with CloudWatch and application logs.

### Additional Sources

- Application logs (structured logs).
- VPC Flow Logs (network activity).
- ALB/ELB access logs (request details).
- RDS logs (database queries and errors).
- S3 server access logs (object-level access).

## 2. CloudWatch Logs Insights

### Example Queries

**Find error messages**

```
fields @timestamp, @message
| filter @message like /ERROR/
| sort @timestamp desc
| limit 50
```

**Find high latency requests**

```
fields @timestamp, @message, latency
| filter latency > 1000
| sort latency desc
| limit 50
```

**Count errors by service**

```
fields @message
| filter @message like /ERROR/
| stats count() by service
| sort count desc
```

## 3. CloudTrail Investigation

### Management vs Data Events

| Event Type | Description | Examples |
|---|---|---|
| Management Events | Show control plane operations on AWS resources. | CreateInstance, DeleteBucket, AttachRolePolicy |
| Data Events | Show resource level activity. | S3 object access, Lambda invocations, DynamoDB reads/writes |

### Key Investigation Steps

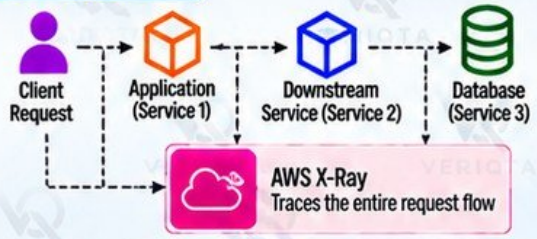
1. Identify the time range of the incident.
2. Look for unusual API calls or unauthorized activity.
3. Check the IAM principal, source IP, and user agent.
4. Investigate failed API calls (error codes).
5. Correlate with CloudWatch logs and other data sources.

### Example CloudTrail Event (Simplified)

```
{
  "eventTime": "2026-10-13T12:34:56Z",
  "eventSource": "ec2.amazonaws.com",
  "eventName": "RunInstances",
  "userIdentity": {
    "type": "AssumedRole",
    "arn": "arn:aws:sts::123456789012:assumed-role
     /DeploymentRole/session"
  },
  "sourceIPaddress": "203.0.113.10",
  "errorCode": "UnauthorizedOperation"
}
```

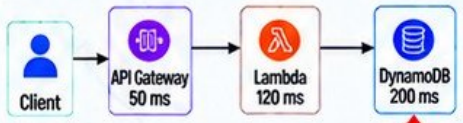
## 4. AWS X-Ray and Distributed Tracing

### How X-Ray Works




### X-Ray Trace Example

Trace Map (Simplified)




### What to Look For

- High latency segments.
- Errors and fault responses.
- Throttling and retries.
- Slow downstream services.
- Cold starts (for Lambda).
- Service dependencies.
- Compare timing with CloudWatch metrics and logs.

## 5. Correlation IDs, Service Dependencies, and Request Reconstruction

### Correlation IDs

- Unique ID for each request across services.
- Helps trace a request end-to-end in logs, traces, and metrics.
- Include it in application logs and propagate across all services.

### Reconstructing a Request

1. Find the correlation ID in application logs.
2. Use CloudWatch Logs, X-Ray, and CloudTrail to follow the request.
3. Identify where the latency or error occurred.
4. Check downstream dependencies (databases, external APIs, etc).

## 6. Production Scenario: Elevated Latency with Healthy CPU and Memory Metrics

### What Happened?

- Users report slow response times.
- CPU and memory metrics look normal.
- No instance failures.
- Investigation shows increased latency in a downstream service and database queries taking longer.

### Investigation Approach

1. Check CloudWatch Logs Insights for slow requests.
2. Use X-Ray to identify the slowest service segment.
3. Review database logs for slow queries.
4. Check for recent changes or external dependency issues.
5. Correlate with CloudTrail for configuration changes.

### Possible Root Causes

- Slow database queries (lock contention or missing index).
- Increased latency from an external API or service.
- Throttling or rate limits.
- Network latency between services.
- Inefficient code path introduced in a recent deployment.

## 7. Senior Engineer Decision

- Use logs and traces to find the true root cause, not just infrastructure metrics.
- Consider both internal and external dependencies.
- Apply targeted fixes (e.g., optimize queries, adjust timeouts, retry logic).
- Communicate impact, resolution steps, and preventive measures to stakeholders.

## 8. Key Takeaways

- Use CloudWatch Logs Insights for fast log analysis.
- Leverage CloudTrail for audit and security investigation.
- Use AWS X-Ray to trace requests and find latency issues.
- Always correlate logs, traces, and metrics.
- Request reconstruction with correlation IDs is essential.
- Do not rely only on CPU and memory metrics.

# INCIDENT COMMAND, ESCALATION, AND COMMUNICATION

## 1. Incident Roles and Responsibilities

### Incident Commander (IC)

- Owns the overall incident response.
- Makes key decisions.
- Coordinates all teams.
- Ensures clear communication.
- Focuses on customer impact and resolution.

### Technical Lead

- Leads technical investigation and resolution.
- Coordinates engineers and SME support.
- Validates and approves production changes.
- Provides technical updates to the IC.

### Communications Lead

- Manages internal and external communications.
- Prepares stakeholder and customer updates.
- Ensures consistent and accurate messaging.
- Works with legal, PR, and support teams when needed.

### Supporting Roles

- SRE / Operations (engineering execution).
- Security (threat analysis).
- Product (customer impact).
- Customer Support (user communication).
- Leadership (business decisions).

## 2. Incident Lifecycle and Timeline

| | | |
|---|---|---|
| 1 | Detect | Alert triggered or issue reported. |
| 2 | Triage | Validate the issue and assess customer impact. |
| 3 | Mobilize | Assign roles and start the incident response. |
| 4 | Investigate | Identify root cause and mitigation options. |
| 5 | Resolve | Implement and verify the fix. |
| 6 | Communicate | Share updates with stakeholders and customers. |
| 7 | Review | Conduct a post-incident review and capture lessons learned. |

## 3. Decision Logs and Evidence Capture

### Incident Timeline

- Record key events with timestamps (UTC).
- Include alerts, decisions, changes, and outcomes.
- Maintain a single source of truth (e.g. shared doc or incident tool).

### Decision Logs

- Capture what decisions were made and why.
- Include who approved the decision.
- Document alternatives considered.
- Help with post-incident reviews and accountability.

### Evidence Capture

- Preserve logs, metrics, traces, and screenshots.
- Keep CloudTrail records.
- Save configuration changes.
- Do not delete or modify evidence during the incident.

## 4. Stakeholder Updates and Customer Communication

### Internal Stakeholders

- Engineering teams or status and next steps.
- Product and business teams for customer impact.
- Leadership for major incidents.
- Use clear, factual, and consistent messaging.

### Customer Communication

- Be transparent about the issue.
- Share impact, current status, and next update time.
- Avoid technical jargon.
- Use status pages, email, or in-app notifications.
- Update regularly, even if there is no new information.

### Communication Best Practices

- One spokesperson for external communication.
- Keep messages consistent across channels.
- Acknowledge impact and give realistic timelines.
- Notify customers when the issue is resolved.

## 5. AWS Support Escalation

### When to Contact AWS Support

- Service disruption or potential AWS service issue.
- Need guidance during complex incidents.
- Suspected AWS infrastructure problem.
- Require service limit increases or technical assistance.

### Support Plan Considerations

- Basic: Customer service and documentation.
- Developer: Business-hours support.
- Business: 24/7 support and faster response times.
- Enterprise: Dedicated TAM, proactive guidance, and fastest response.

### Information to Provide

- Clear description of the issue and impact.
- Affected AWS services, regions, and account ID.
- Relevant CloudTrail event IDs, error messages, and timestamps.
- Troubleshooting steps already taken.

## 6. Coordination to Avoid Conflicting Changes




### Best Practices

- All production changes must be coordinated through the IC.
- Use a change log to track who is making changes.
- Communicate planned changes to all responders.
- Avoid overlapping or conflicting fixes.
- Validate the impact of each change before proceeding.
- Keep a clear record of what was changed, by whom, and when.

## 7. Senior Engineer Decision

- Coordinate multiple responders without creating conflicting production changes.
- Balance speed of resolution with risk management.
- Ensure clear communication across technical, business, and customer stakeholders.
- Know when to escalate to AWS Support or leadership.
- Maintain focus on customer impact and long-term reliability.

## 8. Key Takeaways

- Clear roles and ownership reduce confusion during incidents.
- Detailed timelines, decision logs, and evidence are essential.
- Transparent and consistent communication builds trust.
- AWS Support can provide critical assistance and faster resolution.
- Strong coordination prevents conflicting changes and reduces risk.
- Post-incident reviews help improve future response.

# SAFE MITIGATION, ROLLBACK, AND AUTOMATED RECOVERY

## 1. Observe, Confirm, Change, and Verify

### 1. Observe

- Review alarms, logs, metrics, and user reports.
- Understand the scope and impact.
- Avoid making changes based on assumptions.

### 2. Confirm

- Validate the root cause.
- Reproduce the issue if possible.
- Confirm the change will address the problem.
- Identify potential risks and blast radius.

### 3. Change

- Implement the mitigation or rollback.
- Use a controlled and documented process.
- Follow approval procedures for high-risk changes.

### 4. Verify

- Confirm the issue is resolved.
- Monitor metrics, logs, and user experience.
- Watch for unexpected side effects.
- Keep the incident open until stability is confirmed.

## 2. Deployment Rollback and Feature-Flag Mitigation

### Deployment Rollback

- Revert to the last known good version.
- Use blue/green or canary deployments when available.
- Rollback infrastructure changes (e.g. CloudFormation, Terraform) carefully.
- Verify dependent services are also compatible.

### Feature-Flag Mitigation

- Disable or reduce the impact of the problematic feature.
- Use feature flags to isolate the issue without a full rollback.
- Gradually re-enable after verification.
- Helps mitigate user impact while keeping the system running.

## 3. AWS Systems Manager Automation and Incident Runbooks

### Systems Manager Automation

- Run predefined automation documents.
- Perform common remediation tasks (e.g. restart services, update config, scale resources).
- Reduce manual steps and human error.
- Integrate with IAM for controlled permissions.

### Incident Runbooks

- Step-by-step instructions for common incidents.
- Include troubleshooting steps, commands, and rollback procedures.
- Keep runbooks updated and
- Store runbooks in a central location (e.g. Systems Manager, Wiki, or Git).

## 4. EventBridge-Triggered Remediation and Guardrails

### Automated Remediation Flow

CloudWatch Alarm → Amazon EventBridge → Systems Manager Automation → Remediation Action

### Use Cases

- Automatically restart unhealthy instances.
- Scale services based on alarms.
- Trigger remediation for common failures.
- Notify teams when automated actions run.

### Guardrails

- Use IAM least privilege.
- Require approvals for high-risk actions.
- Implement safeguards to prevent repeated or harmful actions.
- Log all automated actions.
- Use rate limits and conditions to avoid unintended impact.

## 5. Idempotency, Permissions, Approvals, and Rollback Failure Risks

### Idempotency

- Automation should be safe to run multiple times.
- Ensure actions do not create duplicate resources or unintended changes.
- Use idempotent scripts and AWS features.

### Permissions

- Use least privilege IAM roles.
- Grant only the permissions required for the task.
- Avoid using admin credentials in automation.
- Review and rotate permissions regularly.

### Approvals

- Require manual approval for high-risk changes.
- Use Systems Manager Change Manager for approval workflows.
- Define who can approve and under what conditions.

### Rollback Failure Risks

- Rollback may fail due to dependency changes.
- Previous version may be incompatible with current data or configuration.
- Have a plan if rollback does not work.
- Test rollback procedures regularly.

## 6. Production Scenario: Automated Recovery Worsens Outage

### What Happened?

- A service became unhealthy due to a configuration error.
- An automated recovery action (e.g. restart and scale) was triggered by an alarm.
- The action increased load on a dependent database, worsening the outage.
- More instances were started, causing higher traffic and deeper resource exhaustion.

### Key Lessons

- Understand dependencies before automating.
- Use guardrails and rate limits.
- Test automation in non-production environments.
- Monitor downstream services.
- Have a way to quickly stop or roll back automated actions.

## 7. Automated Recovery Workflow with Safeguards

1. **Alarm Triggered** — Metric or health check detects an issue.
2. **Evaluate Conditions** — Check severity, dependencies, and guardrails.
3. **Approval (If Required)** — Automatic or manual approval step.
4. **Execute Automation** — Run Systems Manager Automation.
5. **Verify and Monitor** — Confirm recovery and watch for side effects.

## 8. Key Takeaways

- [x] Follow observe, confirm, change, and verify.
- [x] Use rollback and feature flags to reduce risk.
- [x] Leverage Systems Manager Automation and runbooks.
- [x] Implement EventBridge-triggered remediation with guardrails.
- [x] Ensure idempotency, least privilege, and proper approvals.
- [x] Understand dependencies and test automation.
- [x] Be prepared for rollback failure scenarios.
- [x] Monitor after automated recovery to ensure the issue is resolved.

# DISASTER RECOVERY AND RESILIENCE VALIDATION

## 1. Backup Integrity and Restore Testing

### Why Backups Matter

- Protects against human error, application bugs, and infrastructure failures.
- Enables recovery from ransomware, accidental deletion, and corruption.
- Supports compliance and data retention requirements.

### Backup Best Practices

- Automate backups with AWS Backup.
- Define backup policies and retention periods.
- Encrypt backups and test restores regularly.
- Store backups in a separate account or Region when required.

### Restore Testing

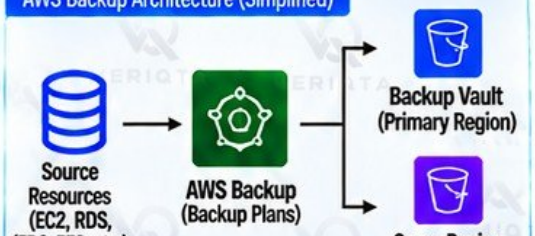
- Regularly perform restore tests.
- Verify data integrity after restore.
- Test application functionality, not just data availability.
- Document results and resolved issues.

### Common Issues

- Backups exist but are untested.
- Inconsistent or incomplete backups.
- Unable to restore dependencies (e.g. IAM roles, KMS keys, or network configuration).
- Corrupted or insufficient backups.

## 2. AWS Backup and Cross-Region Recovery

### AWS Backup Architecture (Simplified)




### Key Capabilities

- Centralized backup management for multiple services.
- Automated backup plans and lifecycle policies.
- Cross-Region and cross-account backups.
- Point-in-time recovery for supported services.
- Integration with AWS Organizations.

## 3. Disaster Recovery Strategies

### Pilot Light

- Minimal core infrastructure in secondary Region.
- Launch additional resources during a disaster.
- Lower cost, moderate recovery time.
- Suitable for less critical workloads.

### Warm Standby

- Scaled-down but fully functional environment in secondary Region.
- Faster recovery time than pilot light.
- Higher cost due to running resources.
- Suitable for critical workloads with moderate RTO/RPO requirements.

### Active-Active

- Fully active environments in multiple Regions.
- Automatic failover and high availability.
- Highest cost and operational complexity.
- Suitable for mission-critical workloads with low RTO/RPO requirements.

## 4. AWS Fault Injection Service (FIS)

### What is FIS?

- Fully managed service to run controlled experiments on AWS infrastructure.
- Helps test resilience and validates recovery procedures.
- Simulates real-world failures in a safe and controlled way.
- Supports experiments on EC2, RDS, ECS, EKS and more.

### Example Experiments

- Terminate EC2 instances.
- Throttle network traffic or introduce latency.
- Stop RDS database instances.
- Simulate service API failures.
- Test auto scaling and failover mechanisms.
- Measure recovery time and validate monitoring and alerts.

## 5. RTO/RPO Validation and Recovery Dependency Testing

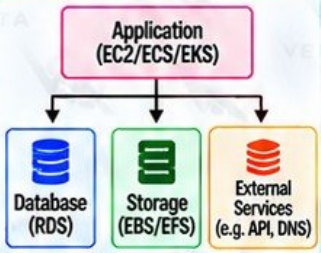
### Key Definitions

- **RTO (Recovery Time Objective):** Maximum acceptable time to restore service after a disaster.
- **RPO (Recovery Point Objective):** Maximum acceptable amount of data loss measured in time.

### Validation Steps

1. Define RTO and RPO for each workload.
2. Map dependencies (databases, services, network, IAM, etc.).
3. Perform end-to-end recovery tests.
4. Measure actual recovery time and data loss.
5. Identify and resolve bottlenecks.
6. Document results and update runbooks.

### Recovery Dependency Example




### Common Dependency Issues

- Missing or untested dependencies.
- Services restored in the wrong order.
- IAM permissions not available during recovery.
- External dependencies (e.g. DNS, third-party APIs) not considered.
- Long recovery times due to manual steps.

## 6. Production Scenario: Recovery Exercise Findings Issues

### Scenario

- A company performs a disaster recovery exercise for a critical application.
- The database is successfully restored in the secondary Region, but the application fails to start.
- The issue is caused by missing IAM permissions and an untested external dependency (DNS records).
- Recovery takes longer than the RTO, and some data is missing beyond the acceptable RPO.

### Investigation and Resolution

1. Identify missing IAM permissions and update roles.
2. Restore and validate DNS configuration.
3. Re-run the recovery steps and verify application health.
4. Update runbooks with the correct order of operations.
5. Implement automated checks for dependencies.

### Key Lessons

- Regular DR exercises reveal real issues that are not visible in design documents.
- Test the full recovery process, including dependencies.
- Validate RTO and RPO with actual tests.
- Keep runbooks updated based on findings.
- Involve all relevant teams in the exercises.

## 7. Senior Engineer Decision

- Prove recoverability through regular, hands-on exercises.
- Do not rely on architecture diagrams alone.
- Ensure all dependencies are tested and documented.
- Balance cost, complexity, and recovery objectives.
- Drive continuous improvement based on exercise results.

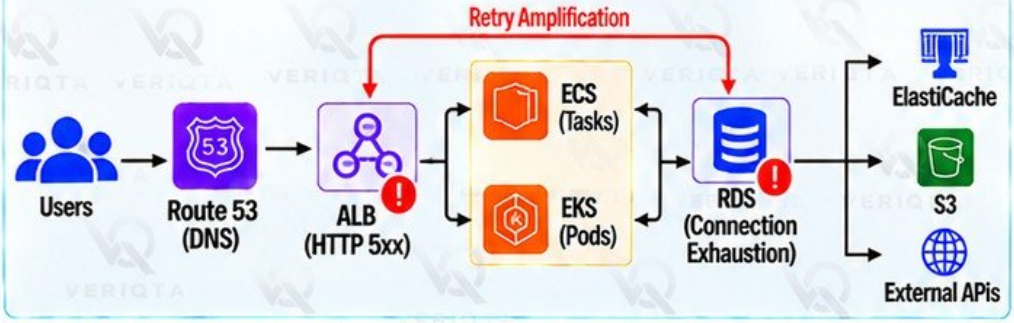
## 8. Key Takeaways

- Backups are only useful if they can be restored.
- Choose a DR strategy that matches business requirements.
- Use AWS Backup and cross-Region replication.
- Leverage AWS Fault Injection Service to test resilience.
- Validate RTO and RPO through real recovery exercises.
- Test dependencies and update runbooks regularly.

# FULL AWS PRODUCTION INCIDENT: INVESTIGATION TO POSTMORTEM

## 1. Incident Scenario Overview (Multi-Service)

A realistic production incident involving multiple AWS services.




## 2. Symptoms and Impact

- ALB returning HTTP 5xx errors.
- Increased request latency.
- ECS/EKS workload instability (restarts and unhealthy tasks/pods).
- RDS connection exhaustion.
- Retry amplification across services.
- Cascading dependency failures (e.g. cache, external APIs).
- Customer-facing errors and service degradation.

## 3. Key Evidence Sources

### Amazon CloudWatch

- ALB 5xx and latency metrics.
- ECS/EKS task or pod metrics.
- RDS connection and CPU metrics.
- Logs (application, container, ALB, and service logs).
- Dashboards and alarms.

### AWS CloudTrail

- API calls and configuration changes.
- Identify unexpected changes during the incident.
- Look for failed API calls and error codes.
- Correlate with IAM activity and assumed roles.

### AWS X-Ray / Tracing

- End-to-end request traces.
- Service dependencies.
- Identify high-latency segments.
- Find errors and throttling.
- Correlate requests using trace IDs and correlation IDs.

### Additional Sources

- Application logs.
- VPC Flow Logs.
- RDS Performance Insights.
- ECS/EKS events and logs.
- External service status.
- On-call notes and timelines.

## 4. Blast-Radius Analysis

- Identify which services are affected.
- Determine customer impact scope.
- Find upstream and downstream dependencies.
- Check which environments are impacted (single service, multiple services, or region-wide).
- Validate if the issue is contained or expanding.

## 5. Investigation and Hypothesis Testing

### 1 Gather Evidence

- Review metrics, logs, traces, and events.
- Build a clear timeline.
- Identify the first signs of the issue.

### 2 Form Hypotheses

- List possible root causes.
- Prioritize based on evidence.
- Validate or eliminate each hypothesis.

### 3 Run Targeted Tests

- Query logs and metrics.
- Check service configurations.
- Compare with the timeline of changes.
- Use tracing to confirm the request flow.

### 4 Identify Root Cause

- Determine the primary root cause.
- Find contributing factors (e.g. retries, timeouts, misconfigurations).
- Understand why existing detection did not prevent impact.

## 6. Safe Mitigation and Service Recovery

- Implement the lowest-risk mitigation first.
- Reduce retry amplification (e.g. backoff, circuit breakers).
- Scale or stabilize ECS/EKS workloads.
- Relieve database pressure (e.g. connection limits, read replicas).
- Recover dependent services in the correct order.
- Monitor key metrics, logs, and user experience.
- Verify service recovery and SLOs.

## 7. Root-Cause Analysis and Corrective Actions

### Root-Cause Analysis

- Write a clear and factual timeline.
- Identify the root cause.
- Document contributing factors (human, process, technical).
- Explain the full sequence of events.
- Include what worked and what did not.

### Corrective Actions

- Implement fixes to prevent recurrence.
- Improve monitoring and alerting.
- Add safeguards (rate limits, circuit breakers).
- Review and improve runbooks.
- Update configurations, capacity, or architecture as needed.

## 8. Blameless Postmortem

- Write a blameless and honest account of the incident.
- Focus on systems, processes, and learnings.
- Include impact, timeline, root cause, contributing factors, and corrective actions.
- Share with relevant stakeholders.
- Turn learnings into long-term improvements.

## 9. Long-Term Reliability Improvements

- Improve system resilience and scalability.
- Strengthen monitoring, alerting, and tracing.
- Reduce single points of failure.
- Validate disaster recovery and failover.
- Run game days and failure simulations.
- Continuously review and update runbooks and operational practices.

## 10. Final Engineering Challenge

- Decide what to investigate first and why.
- Determine which evidence is most reliable.
- Choose the mitigation that presents the lowest risk.
- Define how to verify recovery and confirm SLO restoration.
- Recommend architectural changes to prevent recurrence.

## 11. Key Takeaways

- Use data and evidence, not assumptions.
- Follow a structured investigation process.
- Minimize risk when making changes in production.
- Validate recovery with real user experience and SLOs.
- Turn every incident into long-term reliability improvements.